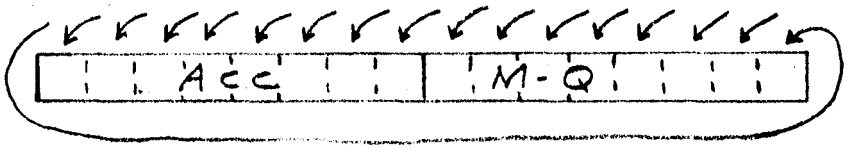
# A Method for doing Fast Binary Division

Dura W. Sweeney Jan 22, 1957

This method will reduce by a factor of two or more the number of divide cycles required to produce a binary quotient. The method is particularly applicable to floating point division where both the divisor and dividend are normal, i.e. There is a one bit in the most significant position of both the divisor and dividend. For fixed point or unnormalized floating point division, the rules are slightly different and will be discussed after the case for normalized floating point division. The method takes advantage of the fact that partial remainders indicate groups of consecutive zeros or ones in the quotient just as some of the fast multiply schemes take advantage of strings of ones and zeros to reduce the number of add cycles. If two numbers are equal for the leading "p" bits and the smaller is subtracted from the larger there will be at least "p" zeros in the partial remainder and if the first bit is inverted to a one, then the first "p" bits of the partial remainder are the same as the first "p" bits of the quotient. If two numbers are equal for the leading "p" bits and the larger is subtracted from the smaller and the partial remainder appears in complement form there will be at least "p" leading ones in this partial remainder. If the first bit is inverted from a one to a zero, then the first "p" bits of the complement partial remainder are the same as the first "p" bits of the quotient.

The following example will illustrate the application of these principles: Assume a register set similar to the 701-704 with a left ring shift ability (see figure). Partial remainders will appear as true or complement results in the



ACC-MQ after combining the divisor with the ACC. Carry out of the high order of the ACC will be detected but not appear in the result. A counter for the number of left ring shift cycles will be provided. Assume a double length dividend in the ACC-MQ or that the MQ is cleared to zero. The equivalent of these registers can be assumed.

First cycle:

1. Subtract divisor from ACC

2a. If no overdraw (true partial remainder) left ring shift ACC-MQ until a one appears in the high order of the ACC. Invert the first bit as it enters the low order of the MQ from a zero to a one. Count the number of ring shifts. Stop the operation if the count equals the size of the quotient to be developed.

2b. If overdraw occurs (complement partial remainder) left ring shift ACC-MQ until a zero appears in the high order of the ACC. Invert the first bit as it enters the low order of the MQ from a one to a zero. Reduce the dividend exponent by one and reduce the ring shift count by one.

Second and subsequent cycles (if necessary):

A. Subtract divisor from ACC if previous partial remainder was true.

1a. If no overdraw (true partial remainder) left ring shift ACC-MQ until a one appears in the high order of the ACC. Invert the first bit as it enters the low order of the MQ from a zero to a one. Add to the ring shift counter the number of positions shifted and stop the operation if the total cound equals the size of the quotient to be developed.

1b. If overdraw (complement partial remainder) left ring shift ACC-MQ until a zero appears in the high order of the ACC. Invert the first bit as it enters the low order of the MQ from a one to a zero. Add to the ring shift counter the number of positions shifted and stop the shift operation if the total count equals the size of the quotient to be developed.

B. Add divisor to the ACC if the previous partial remainder was complement.

1a. If carry (true partial remainder) follow rule for step A 1a above.

1b. If no carry (complement partial remainder) follow rule for step A 1b above.

Final cycle:

When the required number of quotient digits have been developed in the MQ, the divide terminates. If no remainder is desired clear the ACC. If a remainder is desired and the last partial remainder was true, the remainder is in the ACC. If the last partial remainder was complement add the divisor to the ACC until a carry occurs out of the high order of the ACC. Note that one or two add cycles may be necessary.

The quotient exponent is the difference of the dividend less the divisor exponent. The remainder exponent is the difference of the dividend exponent less the ring shift count.

The following numeric examples are for an assumed eight bit register set.

```
Divisor          Dividend
10 10 00 00   11 00 00 11 | 00 11 00 11
             -10 10 00 00
              ----------
              00 10 00 11 | 00 11 00 11
               10 00 11 00 | 11 00 11 | 10      note inverted bit
             -10 10 00 00
              ----------
              11 10 11 00 | 11 00 11 | 10
               01 10 01 10 | 01 1 1 00 11       note inverted bit
             +10 10 00 00
              ----------
              00 00 01 10 | 01 1 1 00 11
               00 11 00 11 | 10 01 1 1 00       note inverted bit, termination in mid-
                                                 shift cycle, and true remainder.
```

```
Divisor          Dividend
11 00 00 00   11 00 00 00 | 00 11 11 11
             -11 00 00 00
              ----------
              00 00 00 00 | 00 11 11 11
               00 11 11 11 | 10 00 00 00
```

```
Divisor          Dividend
10 00 00 01   10 00 00 00 | 00 00 00 00
             -10 00 00 01
              ----------
              11 11 11 11 | 00 00 00 00
               00 00 00 00 | 0 1 11 11 11
             +10 00 00 01
              ----------
              10 00 00 01 | 0 1 11 11 11
               00 00 00 10 | 11 11 11 10       note extra cycle required and
             +10 00 00 01
              ----------                         two add cycles to produce true remainder
               10 00 00 11
             +10 00 00 01
              ----------
               00 00 01 00
```

Fixed point and unnormalized floating point divisors can be handled by one of two methods.

A.

1. Shift the divisor left until a one bit is in the high order position. Count and "remember" the amount of shift. Shift the dividend left the same number of places. Ring shift all remaining zero in the dividend to develop leading quotient zeros.

2. Divide in the manner described above.

3. Shift the divisor and remainder right the amount that the divisor was shifted left before the divide.

B. Count the leading zeros in the divisor without shifting. Then set the carry block in the ACC at that point and consider all ring shifts from

that point to the low order of the MQ. In essence consider both the divisor and ACC reduced in length.

DWS:jdk
1/22/57